

Université Hassan II de Casablanca

Faculté des Sciences Aïn Chock

Module M11 - Automatisation des tests et qualité logicielle

Plan de tests

Plan de tests

Automatisation des tests UI, API, performance et sécurité d'une application web de gestion d'étudiants

Réalisé par : [NOM PRÉNOM / MEMBRES]

Filière : IIIA

Encadrant : [NOM DU PROF]

Année universitaire : 2025-2026

Dépôt Git : github.com/essalhi-moussa/Automatisation_des_Tests_et_Qualite_Logicielle

Table des matières

1. Introduction et objectifs	3
2. Périmètre	3
3. Stratégie de test	3
4. Environnements et outils	4
5. Critères d'entrée et de sortie	4
6. Risques et parades	4
7. Scénarios de test	6
7.1 Interface utilisateur (Selenium)	6
7.2 API (RestAssured et Postman)	7
7.3 Performance et sécurité	8
8. Matrice de traçabilité	10
9. Organisation et livrables	10

1. Introduction et objectifs

Ce document décrit la stratégie, le périmètre et les scénarios de test retenus par l'équipe QA pour l'application web de gestion d'étudiants du projet QAOps. Il sert de référence pour les scripts livrés dans le dépôt Git et pour les rapports d'exécution produits par le pipeline CI/CD.

- Vérifier automatiquement les parcours d'interface essentiels (formulaire, pop-ups, alertes, frames, listes, boutons).
- Valider le contrat des API REST : codes HTTP, contenu JSON, structure (schéma) et temps de réponse.
- Mesurer le comportement sous charge (50 utilisateurs simultanés) : temps de réponse, débit, taux d'erreur.
- Repérer les vulnérabilités simples (en-têtes manquants, cookies, composants obsolètes, XSS) et formuler des recommandations.
- Exécuter l'ensemble dans un pipeline reproductible qui produit des rapports et notifie l'équipe.

2. Périmètre

Dans le périmètre	Hors périmètre
UI : application de démonstration Formy (formulaire, modal, switch-window, dropdown, radio, checkbox, boutons, champs désactivés). API : Reqres.in (/api/users, /api/login). Performance : GET et POST sur Reqres. Sécurité : analyse passive de Formy avec OWASP ZAP.	Tests unitaires du code applicatif (non fourni). Tests d'accessibilité et de compatibilité multi-navigateurs (Chrome uniquement exécuté ; Firefox et Edge sont prévus dans la fabrique de drivers). Analyse active (attaques) d'un site dont nous ne sommes pas propriétaires. Tests de charge au-delà de 50 utilisateurs.

Choix documentés. (1) Formy n'a ni page de connexion ni formulaire d'inscription dédié : la page `/form` (« Complete Web Form ») est utilisée comme scénario d'inscription. (2) Formy n'a aucune page contenant un `iFrame` : l'exigence « frames/iFrames » est couverte par une page locale autonome (déterministe) et par `the-internet.herokuapp.com/nested_frames` (site de démonstration public, exécuté à part car instable).

3. Stratégie de test

La stratégie suit la pyramide des tests : le plus grand nombre de contrôles se fait à la couche API (rapide et stable), l'interface est limitée aux parcours essentiels, et la performance et la sécurité complètent le dispositif.

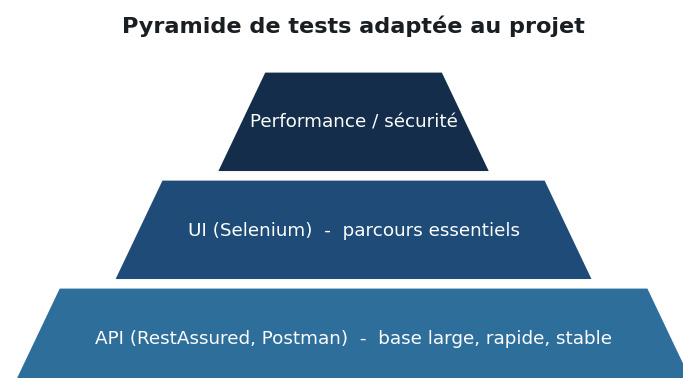


Figure 1 - Pyramide de tests adaptée au projet

Tableau 1 - Niveaux de test

Niveau	Objectif	Outils	Déclenchement
API	Contrat, statuts, schémas JSON, temps de réponse, cas négatifs	RestAssured, TestNG, Postman + Newman	À chaque push (CI)
UI	Parcours utilisateur, pop-ups, alertes, frames, listes	Selenium 4, TestNG, Allure	À chaque push (CI)
Performance	50 utilisateurs, temps, débit, erreurs	JMeter 5.6 (mode CLI, dashboard HTML)	Manuel ou planifié
Sécurité	En-têtes, cookies, composants, XSS	OWASP ZAP (baseline passif)	Planifié, non bloquant

Principes appliqués dans le code : Page Object Model, WebDriver stocké dans un ThreadLocal, attentes explicites uniquement (aucun `Thread.sleep` dans les pages), données de test fournies par des data providers TestNG, capture d'écran jointe à Allure en cas d'échec, URL et clé API centralisées dans `config.properties`.

4. Environnements et outils

Tableau 2 - Environnement de test

Élément	Valeur
Poste d'exécution	Windows 11, Java 21 (compilation en cible Java 17), Maven 3.9, Chrome (version courante) via Selenium Manager
Applications cibles	Formy (formy-project.herokuapp.com) ; Reqres (reqres.in) ; the-internet.herokuapp.com (frames)
Mock local	mock/reqres-mock.js (Node.js) : reproduit le contrat observé de Reqres, sert à développer sans consommer le quota
Tests UI / API	Selenium 4.35, TestNG 7.10, RestAssured 5.5 + json-schema-validator, AssertJ, Allure TestNG 2.29
API (Postman)	Collection v2.1 + environnements ; Newman avec reporter htmlextra
Performance	Apache JMeter 5.6.3, plan jmeter/reqres-load-test.jmx
Sécurité	OWASP ZAP 2.17.0 (plan Automation Framework équivalent à zap-baseline.py ; image Docker en CI)
CI/CD	GitLab CI (.gitlab-ci.yml) ; Jenkinsfile déclaratif en alternative

5. Critères d'entrée et de sortie

Tableau 3 - Critères

Critères d'entrée	Critères de sortie
Dépôt Git à jour et compilable (<code>mvn test-compile</code>). Applications cibles joignables (HTTP 200). Quota Reqres disponible, sinon utilisation du mock local. Outils installés (Chrome, JMeter, Newman, ZAP).	100 % des tests de la suite <code>ui</code> et de la suite <code>api</code> réussis. Collection Newman : 0 assertion en échec. Test de charge exécuté avec 50 utilisateurs ; taux d'erreur et percentiles rapportés (les 429 sont analysés, pas masqués). Rapport ZAP produit ; aucune alerte de risque élevé non traitée. Rapports archivés dans <code>reports/</code> .

6. Risques et parades

Tableau 4 - Registre des risques

Risque	Impact	Parade
Limite de débit de Reqres : 20 requêtes/minute et quota journalier de 40 requêtes (en-têtes <code>Ratelimit-*</code> , <code>X-Ratelimit-*</code>)	Réponses 429, tests API et test de charge faussés	Gestionnaire de 429 dans les tests (attente puis relance), mock local, analyse du 429 comme résultat

Risque	Impact	Parade
Sites de démonstration instables (the-internet : chargements > 30 s)	Tests UI intermittents	Page locale déterministe pour la couverture iFrame ; tests du site tiers isolés dans le groupe external avec relance limitée
Reqres ne persiste pas les écritures (POST/PUT/DELETE)	Impossible de relire une ressource créée	Assertions sur l'écho de la réponse, pas sur l'état
Pas de Docker sur le poste d'exécution	Scan ZAP par l'image officielle impossible localement	ZAP installé localement avec un plan équivalent ; script Docker utilisé en CI
Évolution des sites tiers (identifiants, contenus)	Échecs non liés à nos scripts	Locators sur identifiants stables, constat documenté dans le rapport

7. Scénarios de test

Chaque scénario précise ses préconditions, ses étapes, son jeu de données, le résultat attendu et le critère de validation. La colonne « Script » renvoie au code automatisé.

7.1 Interface utilisateur (Selenium)

Tableau 5 - Scénarios UI

ID	Titre	Préconditions	Étapes	Jeu de données	Résultat attendu	Critère de validation	Prio.	Script
UI-01	Inscription valide (formulaire complet)	Navigateur ouvert, /form accessible	1. Ouvrir /form 2. Saisir prénom, nom, poste 3. Choisir niveau d'études, sexe, expérience, date 4. Cliquer sur Submit	3 jeux : Yassine Essalhi / Salma Benani / Omar El Idrissi (voir TestData)	Page /thanks affichée avec le message de succès	Titre « Thanks for submitting your form » et alerte « The form was successfully submitted! »	Haute	FormTests.inscriptionValide
UI-02	Soumission avec champs vides	/form accessible	1. Ouvrir /form 2. Cliquer sur Submit sans rien saisir	Aucune donnée	Comportement réel de Formy : la soumission aboutit (pas de validation côté client)	Message de succès affiché (constat documenté dans le rapport final)	Moyenne	FormTests.soumissionFormulaireVide
UI-03	Liste déroulante « années d'expérience »	/form accessible	1. Ouvrir /form 2. Sélectionner chaque option du menu	0-1, 2-4, 5-9, 10+	L'option choisie devient l'option sélectionnée	getFirstSelectedOption = libellé choisi	Moyenne	FormTests.listeDeroulanteExperience
UI-04	Ouverture et fermeture du modal (Close)	/modal accessible	1. Ouvrir /modal 2. Cliquer sur « Open modal » 3. Cliquer sur Close	-	Le pop-up apparaît puis disparaît	Modal invisible au départ, visible avec le titre « Modal title », invisible après Close	Haute	ModalTests.ouvrirPuisFermerLeModal
UI-05	Bouton OK du modal	/modal accessible	1. Ouvrir le modal 2. Cliquer sur OK 3. Cliquer sur Close	-	OK n'a pas de gestionnaire (pas de data-dismiss) : le modal reste ouvert	Modal toujours affiché après OK, fermé après Close	Basse	ModalTests.boutonOkSansEffetDeFermeture
UI-06	Alerte JavaScript	/switch-window accessible	1. Ouvrir /switch-window 2. Cliquer sur « Alert » 3. Lire puis accepter l'alerte	-	Alerte « This is a test alert! » acceptée	Texte de l'alerte conforme, page toujours utilisable	Haute	SwitchWindowTests.alerteJavaScriptAcceptee
UI-07	Nouvel onglet	/switch-window accessible	1. Cliquer sur « New Tab » 2. Basculer sur le nouvel onglet 3. Le fermer et revenir	-	Deux fenêtres puis retour à une seule	2 handles après le clic, URL du nouvel onglet = site Formy, retour sur /switch-window	Moyenne	SwitchWindowTests.nouvelOnglet
UI-08	Navigation par le menu déroulant	/dropdown accessible	1. Déplier « Dropdown button » 2. Choisir une entrée	Buttons, Checkbox, Radio Button, Modal	Navigation vers la page choisie	URL contient le fragment attendu ; menu de 10 entrées ou plus	Moyenne	ElementsTests.menuDeroulantNavigation
UI-09	Boutons radio exclusifs	/radiobutton accessible	1. Vérifier l'option 1 cochée par défaut 2. Cocher l'option 3	Options 1 à 3	Une seule option sélectionnée à la fois	Option 3 cochée, options 1 et 2 décochées	Haute	ElementsTests.boutonsRadioExclusifs
UI-10	Cases à cocher indépendantes	/checkbox accessible	1. Cocher les cases 1 et 3 2. Décocher la case 1	Cases 1 à 3	Chaque case change d'état indépendamment	États conformes après chaque action	Moyenne	ElementsTests.casesACocher
UI-11	Boutons et groupe de boutons	/buttons accessible	1. Contrôler les boutons colorés 2. Cliquer sur « Success » 3. Déplier le groupe avec menu	-	Tous les boutons sont actifs ; le menu du groupe propose 2 liens	6 boutons ou plus, tous activés, 2 entrées de menu	Moyenne	ElementsTests.boutons
UI-12	Champ actif / champ désactivé	/enabled accessible	1. Contrôler l'état des deux champs 2. Saisir un texte dans le champ actif	« texte de test »	Le champ désactivé n'est pas modifiable	isEnabled = false (désactivé), true (actif), valeur saisie relue	Basse	ElementsTests.champsActifEtDesactive

ID	Titre	Préconditions	Étapes	Jeu de données	Résultat attendu	Critère de validation	Prio.	Script
UI-13	Frames imbriqués (site tiers)	the-internet.herokuapp.com joignable	1. Ouvrir /nested_frames 2. Entrer dans frame-top puis frame-left/middle/right 3. Lire le texte	LEFT, MIDDLE, RIGHT	Chaque frame contient son libellé	Texte lu = texte attendu (3 frames sur 3)	Moyenne	FramesTests.lireFramesDuHaut (groupe external)
UI-14	Frame du bas puis retour au contenu principal (site tiers)	the-internet.herokuapp.com joignable	1. Lire frame-bottom 2. Revenir au contenu par défaut 3. Lire le frame du milieu	BOTTOM, MIDDLE	Le changement de contexte entre frames fonctionne	Textes lus conformes	Basse	FramesTests.lireFrameDuBas (groupe external)
UI-15	iFrame et iFrame imbriqué (page locale déterministe)	Aucune (page data: autonome)	1. Ouvrir la page locale 2. Lire l'iFrame externe 3. Lire l'iFrame imbriqué 4. Revenir à la page parente	Contenu externe / Contenu imbriqué	Les deux niveaux d'iFrame sont lisibles	Textes conformes, titre parent relu	Haute	FramesTests.lireIframesLocaux

7.2 API (RestAssured et Postman)

Tableau 6 - Scénarios API

ID	Titre	Préconditions	Étapes	Jeu de données	Résultat attendu	Critère de validation	Prio.	Script
API-01	GET /api/users?page=2	API joignable, en-tête x-api-key	GET /api/users?page=2	page=2	200, 6 utilisateurs, pagination cohérente	Statut 200 ; page=2 ; per_page=6 ; data de 6 éléments ; schéma users-list ; durée < 3 s	Haute	UsersApiTests.listerLesUtilisateurs / Postman API-01
API-02	GET /api/users/2	idem	GET /api/users/2	id=2	200, utilisateur Janet Weaver	Statut 200 ; first_name/last_name ; schéma user-single ; durée < 3 s	Haute	UsersApiTests.consulterUnUtilisateur / Postman API-02
API-03	GET /api/users/23 (inexistant)	idem	GET /api/users/23	id=23	404 et corps JSON vide	Statut 404 ; pas de clé data ; durée < 3 s	Haute	UsersApiTests.utilisateurInexistant / Postman API-03
API-04	POST /api/users	idem	POST /api/users avec name et job	morpheus/leader ; Yassine Essalhi/Ingénieur QA (Postman : valeurs aléatoires)	201, données renvoyées avec id et createdAt (non persisté)	Statut 201 ; name/job identiques ; id et createdAt présents ; schéma user-created	Haute	UsersApiTests.creerUnUtilisateur / Postman API-04
API-05	PUT /api/users/2	idem	PUT /api/users/2 avec name et job	morpheus/zion resident	200 avec updatedAt	Statut 200 ; valeurs mises à jour ; updatedAt présent ; schéma user-updated	Moyenne	UsersApiTests.modifierUnUtilisateur / Postman API-05
API-06	DELETE /api/users/2	idem	DELETE /api/users/2	id=2	204 sans contenu	Statut 204 ; corps vide ; durée < 3 s	Moyenne	UsersApiTests.supprimerUnUtilisateur / Postman API-06
API-07	POST /api/login valide	idem	POST /api/login	eve.holt@reqres.in / cityslicka	200 avec un token	Statut 200 ; token non vide ; schéma login-success	Moyenne	UsersApiTests.connexionValide / Postman API-07
API-08	POST /api/login sans mot de passe	idem	POST /api/login avec email seul	peter@klaven	400 « Missing password »	Statut 400 ; message exact ; schéma error	Haute	UsersApiTests.connexionSansMotDePasse / Postman API-08

7.3 Performance et sécurité

Tableau 7 - Scénarios de performance et de sécurité

ID	Titre	Préconditions	Étapes	Jeu de données	Résultat attendu	Critère de validation	Prio.	Script
PERF-01	Charge nominale : 50 utilisateurs, GET et POST	JMeter 5.6 installé ; cible joignable	Lancer scripts/run-jmeter.sh : 50 threads, montée en charge 10 s, 10 boucles ; GET /api/users?page=2 puis POST /api/users (CSV)	jmeter/data/users.csv (40 lignes)	Aucune erreur ; temps de réponse stables	Erreurs < 1 % ; P95 < 3 000 ms ; débit relevé	Haute	jmeter/reqres-load-test.jmx
PERF-02	Comportement face à la limitation de débit (HTTP 429)	Quota Reqres entamé ou épuisé	Rejouer PERF-01 sur l'API publique et observer les codes de réponse	idem	Les 429 sont mesurés et analysés, pas masqués	Part de 429 rapportée dans le tableau de bord et dans le rapport final	Haute	reports/perf/reqres
SEC-01	Analyse passive ZAP de Formy (baseline)	ZAP 2.17 ; cible joignable	scripts/zap-scan.sh : spider (3 min max) puis analyse passive	https://formy-project.herokuapp.com	Rapport des alertes avec niveau de risque	Rapports HTML et JSON produits ; chaque alerte classée (OWASP Top 10 / CWE)	Haute	scripts/zap-scan.sh
SEC-02	Recherche de réflexion XSS sur /form et /thanks	Aucune	Envoyer un marqueur inoffensif dans les paramètres d'URL et rechercher sa réflexion dans la réponse	ZAPXSSEMARK12345	Marqueur non réfléchi	0 occurrence du marqueur dans la page retournée	Moyenne	Contrôle manuel (curl), voir rapport

8. Matrice de traçabilité

Tableau 8 - Exigence vers test

Exigence	Description (sujet)	Scénarios / livrables
E1	Scénario formulaire login / inscription (Formy)	UI-01, UI-02, UI-03
E2	Scénarios boutons et pop-ups (Formy)	UI-04, UI-05, UI-11
E3	Alertes JavaScript et nouvelles fenêtres	UI-06, UI-07
E4	Frames / iFrames	UI-13, UI-14, UI-15
E5	Listes déroulantes et boutons radio	UI-03, UI-08, UI-09, UI-10
E6	API : GET /users, POST /users, PUT /users/{id}, DELETE /users/{id}	API-01 à API-06
E7	Cas négatifs et vérification du schéma JSON	API-03, API-08 + schémas
E8	Postman exporté et exécuté avec Newman	API-01 à API-08 (collection)
E9	Performance : au moins 50 utilisateurs, GET et POST	PERF-01, PERF-02
E10	Sécurité : scan ZAP, XSS, en-têtes manquants	SEC-01, SEC-02
E11	Rapports Allure (UI/API), Newman, JMeter, ZAP	reports/
E12	Pipeline CI/CD (UI + API, rapports, notifications)	.gitlab-ci.yml, Jenkinsfile

9. Organisation et livrables

Tableau 9 - Livrables

Livable	Emplacement
Scripts de test (UI, API), suites TestNG	src/test/java, src/test/resources/testng
Collection Postman exportée et environnements	postman/
Plan de charge JMeter et jeu de données	jmeter/
Scripts ZAP, Newman, JMeter, mock	scripts/, mock/
Pipeline CI/CD	.gitlab-ci.yml, Jenkinsfile
Rapports d'exécution (UI, API, performance, sécurité)	reports/
Plan de tests et rapport final (Word et PDF)	docs/